中南大学 计算机学院
CENTRAL SOUTH UNIVERSITY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

自 然 语 言 处 理

第2章 NLP发展的8个里程碑概述

授课教师：李芳芳
联系方式：lifangfang@csu.edu.cn

1.4 发展历史

自然语言处理发展复苏期的8个里程碑

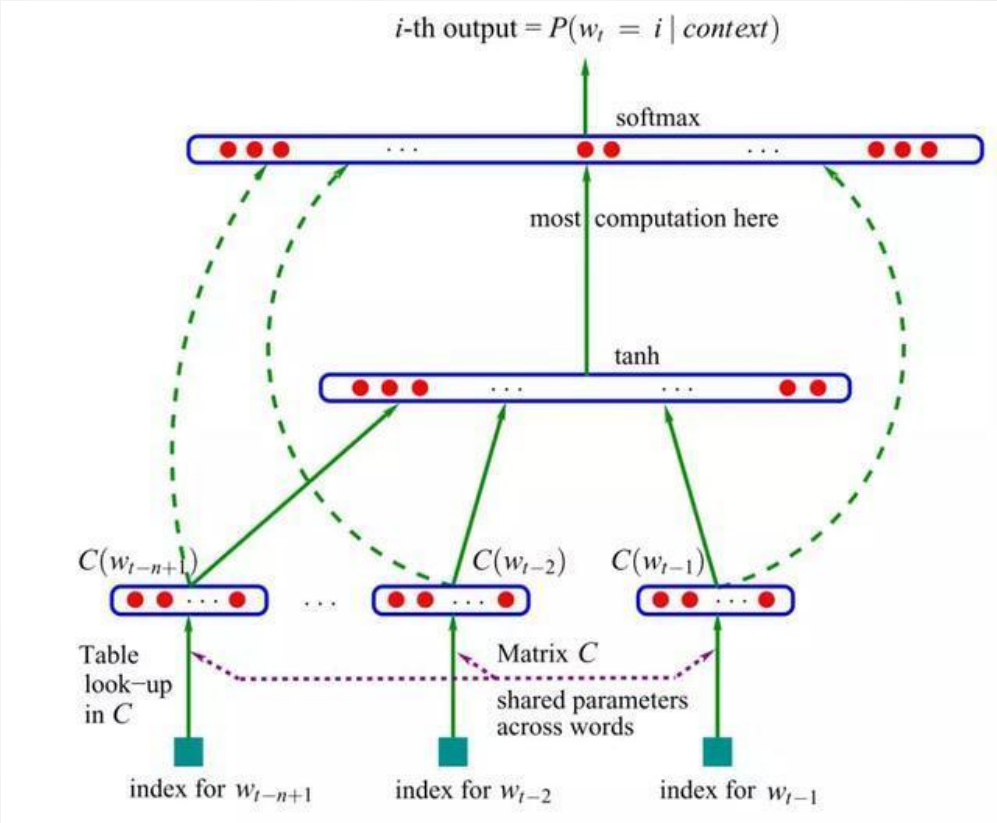
1、2001年——神经语言模型（NEURALLANGUAGE MODELS）

- 语言模型解决的是**在给定已出现词语的文本中，预测下一个单词**的任务。
- 第一个神经语言模型使用了前馈神经网络，是Bengio 等人于 2001 年提出的，以某词语之前出现的n个词语作为输入向量。
- 今天，这样的向量被称为大家熟知的词嵌入（word embeddings）。

1.4 发展历史

神经语言模型

第一个神经语言模型，前馈神经网络，是Bengio 等人于 2001年提出的。
这个模型以某词语之前出现的n个词语作为输入向量，**预测第n个词**。



1.4 发展历史

🔍 **模型结构逐层拆解**

1. 输入层：词索引与词嵌入

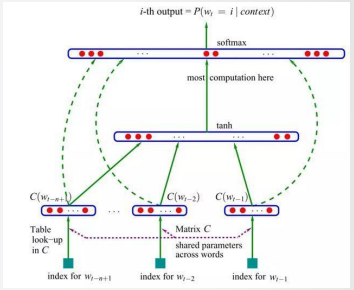
- 输入：固定窗口大小的上下文词索引（如 $w_{t-n+1}, \ldots, w_{t-2}, w_{t-1}$）
- 词嵌入层（矩阵 C）：通过 `Table look-up in C` 查表操作，将每个词索引映射为低维实值向量 $C(w_i)$，即**词向量 / 词嵌入**
- 特点：所有词共享同一个嵌入矩阵 C，实现参数共享，解决传统 n-gram 模型的维度灾难问题

2. 隐藏层：非线性变换

- 输入：将上下文词的词向量拼接为一个长向量
- 变换：通过 `tanh` 激活函数做非线性变换，捕捉词之间的语义和语序关系
- 作用：将高维稀疏输入映射到低维稠密空间，学习语言的内在规律

3. 输出层：概率预测

- 变换：隐藏层输出经过 `softmax` 归一化
- 输出：得到词汇表中每个词作为下一个词 w_t 的概率分布
- 公式：$i\text{-th output} = P(w_t = i \mid context)$，即给定上下文时第 i 个词的预测概率
- 计算特点：`most computation here` 标注说明，softmax 层是模型计算量最大的部分



代码示例（Python）：

python 运行

```python
import numpy as np

# 1. 定义嵌入矩阵C：V=5（词汇表5个词），d=3（词向量维度3）
# 矩阵C是可学习的参数，初始随机初始化，训练中不断更新
C = np.random.randn(5, 3)  # 5行3列的随机矩阵
print("嵌入矩阵C：")
print(C)

# 2. 词汇表（词→索引映射）
vocab = {"<PAD>": 0, "我": 1, "爱": 2, "学习": 3, "AI": 4}

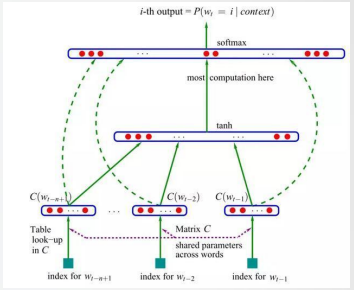
# 3. Table look-up 查表：通过索引取词向量
word = "爱"
word_idx = vocab[word]  # 拿到"爱"的索引：2
word_vec = C[word_idx]  # 取C的第2行，就是"爱"的词向量
print(f"\n'{word}'的索引：{word_idx}")
print(f"'{word}'的词向量：{word_vec}")
```

输出示例：

plaintext

```
嵌入矩阵C：
[[ 0.12  0.34 -0.56]
 [ 0.78 -0.21  0.43]
 [ 0.90  0.11 -0.77]  # "爱"的词向量（第2行）
 [-0.33  0.88  0.22]
 [ 0.55 -0.44  0.66]]

'爱'的索引：2
'爱'的词向量：[ 0.90  0.11 -0.77]
```

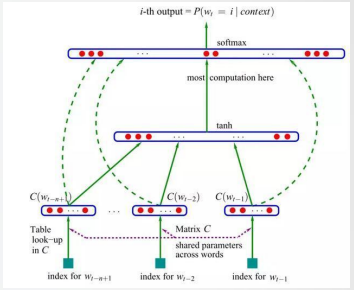


1.4 发展历史

tanh（双曲正切函数）是隐藏层的非线性激活函数，公式很简单：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

它的作用是给模型加入非线性能力（没有非线性，多层神经网络就退化成单层线性模型了）。



在 NNLM 隐藏层中，TANH 层其实是「线性变换 + TANH激活」的组合,第一步涉及加权求和

softmax 层的完整计算步骤：

1. **线性变换**：计算每个词的得分 $z_i = W_i^T \cdot h + b_i$（$W_i$ 是W的第i列，h是隐藏层输出）

   等价于矩阵运算：$z = W^T \cdot h + b$（z是长度为V的得分向量）

2. **指数运算**：对每个得分做指数：$\exp(z_i)$

3. **归一化**：计算每个词的概率：$p_i = \frac{\exp(z_i)}{\sum_{j=1}^{V} \exp(z_j)}$

1.4 发展历史

自然语言处理发展复苏期的8个里程碑

2、2008年——多任务学习（Multi-task learning）

- 多任务学习是在多个任务下训练的模型之间**共享参数的方法**。在神经网络中，这可以通过绑定不同层的权重来轻松完成。
- 在这一框架下，**词嵌入矩阵**被**两个或多个**在**不同任务下训练的模型共享**。共享单词嵌入使模型能够在单词嵌入矩阵中协作和共享一般的低级信息，这通常构成模型中最大数量的参数。
- 它引领了诸如预训练单词嵌入和使用卷积神经网络（CNN）之类的方法，这些方法仅在过去几年中被广泛采用。

多任务学习

2、2008年——多任务学习（MULTI-TASKLEARNING）

- 核心一句话：让一个NLP模型同时学多个相关任务，互相“借力”，最终让模型在**主任务上**的表现更好，就像学生同时学语文、阅读、写作，语文能力会整体提升，而不是单独学某一项。

- 先看“单任务学习”
- 训练一个模型只做一件事，比如只学“情感分析”（判断句子是好评/差评），模型只接触情感分析的数据集，学完后只会干这活，泛化能力弱（遇到类似但不同的问题就不会了）。

1.4 发展历史

多任务学习

2、2008年——多任务学习（Multi-tasklearning）

- 再看“多任务学习”
- 让一个模型同时学多个关联的NLP小任务（叫“辅助任务”），再主攻一个核心任务（叫“主任务”）。辅助任务帮模型学懂语言的通用规律（比如词的意思、句子结构），主任务再把这些规律用在具体问题上。

训练数据（仅情感分析）→ 模型训练 → 只会做【情感分析】

VS

训练数据（主任务+多个辅助任务）→ 一个模型同时训练 → 主任务表现更好+模型更“聪明”

1.4 发展历史

多任务学习

2、2008年——多任务学习（MULTI-TASKLEARNING）

- 举例
- 主任务：判断电商评论的情感（好评/差评）
- 选3个辅助任务：
  - 1. 词性标注（标出每个词是名词/动词/形容词，比如“好用”是形容词）；
  - 2. 实体识别（找出句子里的商品名，比如“这款洗面奶超好用”里的“洗面奶”）；
  - 3. 句子分类（判断句子是陈述/感叹，比如“太香了！”是感叹句）

1.4 发展历史

多任务学习

2、2008年——多任务学习（MULTI-TASKLEARNING）

- 模型同时学这4个任务时，会发现：
  - - 形容词“好用”、“差”常和情感相关；
  - - 商品名附近的词往往是评价核心；
  - - 感叹句大概率情感更强烈。
  - 这些通用规律被模型记住后，再做「情感分析」主任务，准确率会比单独学高很多！

1.4 发展历史

多任务学习

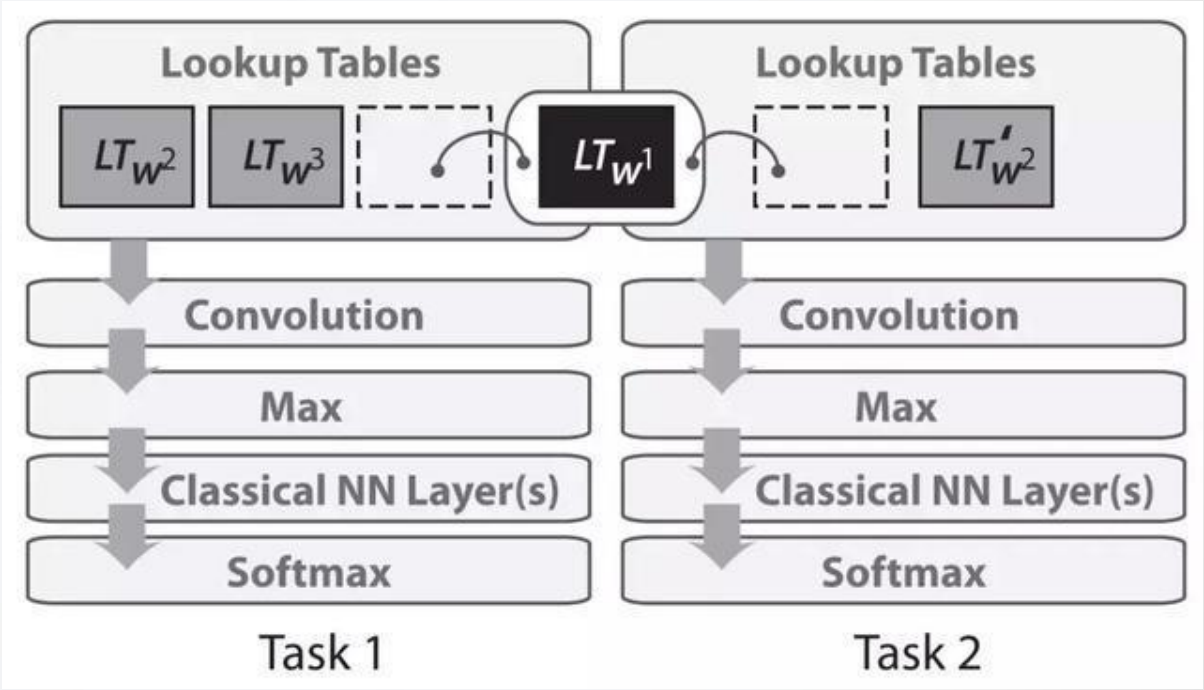
2、2008年——多任务学习（MULTI-TASKLEARNING）

- 把NLP模型比作学做菜的厨师：
- - 单任务学习：只学做“番茄炒蛋”，只会这一道菜，换个“番茄炖牛腩”就不会了；
- - 多任务学习：同时学“番茄炒蛋”（主任务）+“切番茄/打鸡蛋”（辅助任务1）+“调咸淡味”（辅助任务2）+“炒菜的火候”（辅助任务3）。
- 学完后，不仅番茄炒蛋做得更好，还能举一反三做番茄炒土豆、辣椒炒蛋，厨艺整体提升——这就是多任务学习的核心！

1.4 发展历史

多任务学习总结

- 词嵌入矩阵被两个或多个在不同任务下训练的模型共享。
- 共享的词嵌入矩阵使模型可以**相互协作**，**共享矩阵中的低层级信息**，而词嵌入矩阵往往构成了模型中需要训练的绝大部分参数。



1.4 发展历史

- NLP多任务学习的核心优势（3个关键点）
  - 1. 提升主任务效果：辅助任务帮模型掌握语言底层逻辑，避免“学死知识”，泛化能力更强（遇到没见过的句子也能判断）。
  - 2. 节省资源：一个模型搞定多个事，不用为每个任务单独训练，省时省算力。
  - 3. 缓解数据不足：如果主任务的训练数据很少（比如小众领域的情感分析），辅助任务的海量通用数据能“补位”，让模型不练偏。

- 常见的NLP多任务学习类型（简单分2类）
  - 1. 共享底层+任务专属顶层（最常用）
    - 模型底层的网络（负责学通用语言特征，比如词向量、句子结构）是所有任务共享的，顶层再分不同的小网络，对应不同的任务。
  - 2. 多任务轮流训练
    - 把多个任务的训练数据混在一起，模型每次随机学一个任务的样本，轮流提升各任务能力，适合任务关联性较强的场景。

1.4 发展历史

NLP多任务学习不是“贪多嚼不烂”，而是利用任务间的关联性，让模型学透语言的通用规律，最终实现“1+1>2”的效果；

是现在训练优秀NLP模型（比如ChatGPT、文心一言等大模型）的核心方法之一；

大模型其实就是同时学了上亿个NLP相关任务，才拥有了强大的语言理解和生成能力。

1.4 发展历史

多任务学习总结

- 多任务学习于 2008 年被COLLOBERT 和 WESTON 等人首次在自然语言处理领域应用于神经网络。
- 多任务学习通过在多个任务下训练的模型之间共享参数的方法，鼓励模型学习对多个任务有效的表征描述。
- 这对于学习一般的、低级的描述形式、集中模型的注意力或在训练数据有限的环境中特别有用。
- 如今，多任务学习在自然语言处理领域仍被广泛使用，已经成为NLP任务中的一个非常有用工具。

1.4 发展历史

TIPS：过渡

- 虽然神经网络语言模型解决了传统语言模型中维度灾难和数据稀疏性的问题，并且相较于传统语言模型，性能有了很大的提升。
- 但其在计算词向量的过程中计算量太大、太复杂。
- 因此，在2013年，MIKOLOV 等人提出了WORD2VEC，通过去除隐藏层和近似计算目标使词嵌入的训练更为高效，使得大规模的词嵌入模型训练成为可能。

1.4 发展历史

自然语言处理发展复苏期的8个里程碑

3、2013年——词嵌入（Word Embeddings）

- 词嵌入在2001年首次出现。而Mikolov等人在2013年作出的主要创新——是通过删除隐藏层和近似目标来使这些单词嵌入的训练更有效。
- 虽然这些变化本质上很简单，但它们与高效的word2vec（word to vector，用来产生词向量的相关模型）组合在一起，**使得大规模的词嵌入模型训练成为可能**。

1.4 发展历史

词嵌入

- 通过稀疏向量对文本进行表示的**词袋模型**，在自然语言处理领域已经有很长的历史了。
- 而用稠密的向量对词语进行描述，则最近几年才开始。2013 年MIKOLOV等人工作的主要创新之处在于，通过**去除隐藏层和近似计算**目标使词嵌入模型的训练更为高效。
- 尽管这些改变在本质上是十分简单的，但它们与高效的 WORD2VEC（WORD TO VECTOR，用来产生词向量的相关模型）组合在一起，使得**大规模的词嵌入**模型训练成为可能。

1.4 发展历史

词嵌入：词集模型

- 最容易想到的方法是：统计各词在文本中是否出现，出现则表示为 1，未出现则表示为 0，即one-hot编码 。
- One-hot编码：又称独热编码，用N位状态寄存器编码N个状态，每个状态都有独立的寄存器位，且这些寄存器位中只有一位有效（只能有一个状态）。
- 词集模型：使用one-hot编码对文本进行向量化。

1.4 发展历史

词嵌入：词集模型

EXAMPLE

待向量化词串： 自然语言 理解

文本全部词汇：机器、学习、自然语言、处理、理解

向量化结果： [0, 0, 1, 0, 1]

词嵌入：词袋模型

- 词集模型只统计词是否出现，无法有效表示词的重要性信息。很容易想到，可以根据词频（Term Frequency）来判断词的重要性：如果一个词（非停用词）在文本中出现的次数越多，那么该词对于文本来说越重要。

- **词袋模型（Bag Of Word）**：在词集模型的基础上，统计各词在文本中出现次数。（文档中出现的词对应的one-hot向量相加）

1.4 发展历史

词嵌入：词袋模型

思考

在统计指标上
还可以做哪些改进？

词频 → 进一步处理 → ???

1.4 发展历史

词嵌入：离散化表示

- 离散化表示方法是最早的以词语为基础处理单元的文本向量化方法。这类方法虽然简单易行，但是普遍存在如下三个方面的问题：**维度灾难，无法保留词序信息，存在语义鸿沟。**

- 由于离散化表示方法存在以上不足以及神经网络的再次流行，故近年来的研究重心转移到**利用无标注数据挖掘有价值的信息上来，**被广泛使用的Word2Vec就是利用神经网络，从大量无标注的文本中提取有用的信息。

词嵌入

- 思路：通过训练，**将每个词都映射到一个较短的词向量上来**，这个过程被称为**词嵌入（word embedding）**。所有的这些词向量就构成了向量空间，进而可以用普通的统计学的方法来研究词与词之间的关系。

注意：

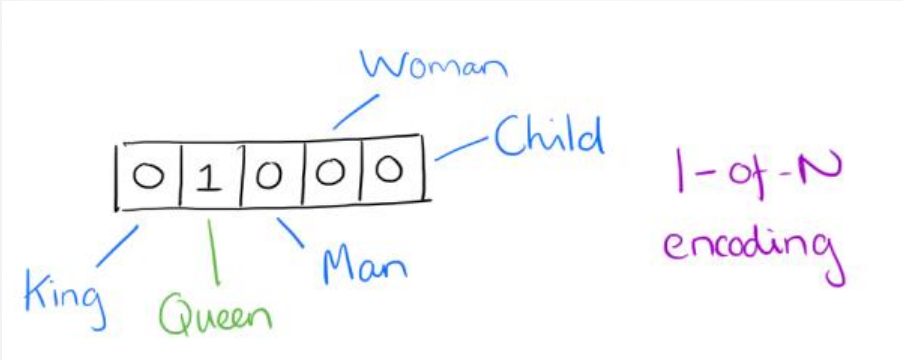
1、较短的词向量的维度往往需要在训练时由自己来指定。

2、映射之后的向量应尽量避免丢失之前的向量所含的信息。

1.4 发展历史

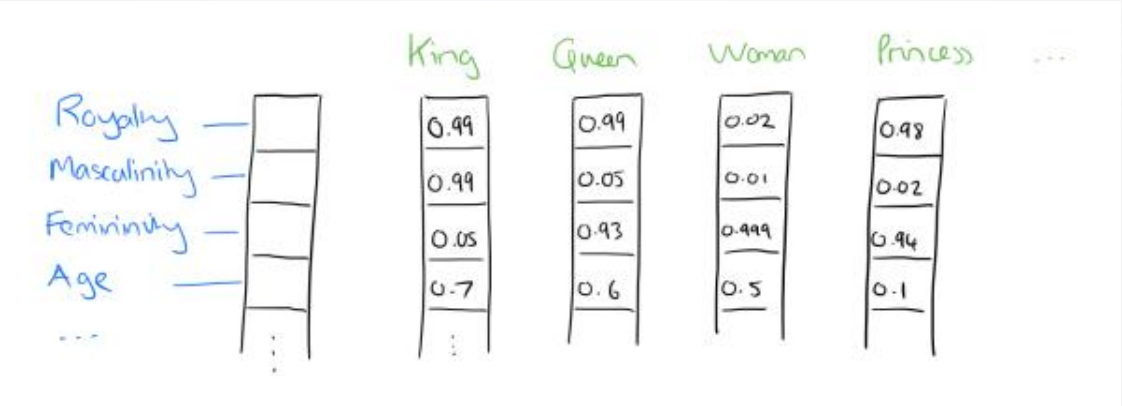
词嵌入：One-hot编码

- 有下面的5个词组成的词汇表，词"Queen"的序号为2， 那么它的词向量就是(0,1,0,0,0)。
- 同样的道理，词"Woman"的词向量就是(0,0,0,1,0)。这种词向量的编码方式叫做one hot representation.



词嵌入：Distributed 编码

- 比如下图我们将词汇表里的词用"Royalty","Masculinity", "Femininity"和"Age"4个维度来表示。
- King这个词对应的词向量可能是**(0.99,0.99,0.05,0.7)**。当然在实际情况中，我们**并不能**对词向量的每个维度做一个**很好的解释**。



1.4 发展历史

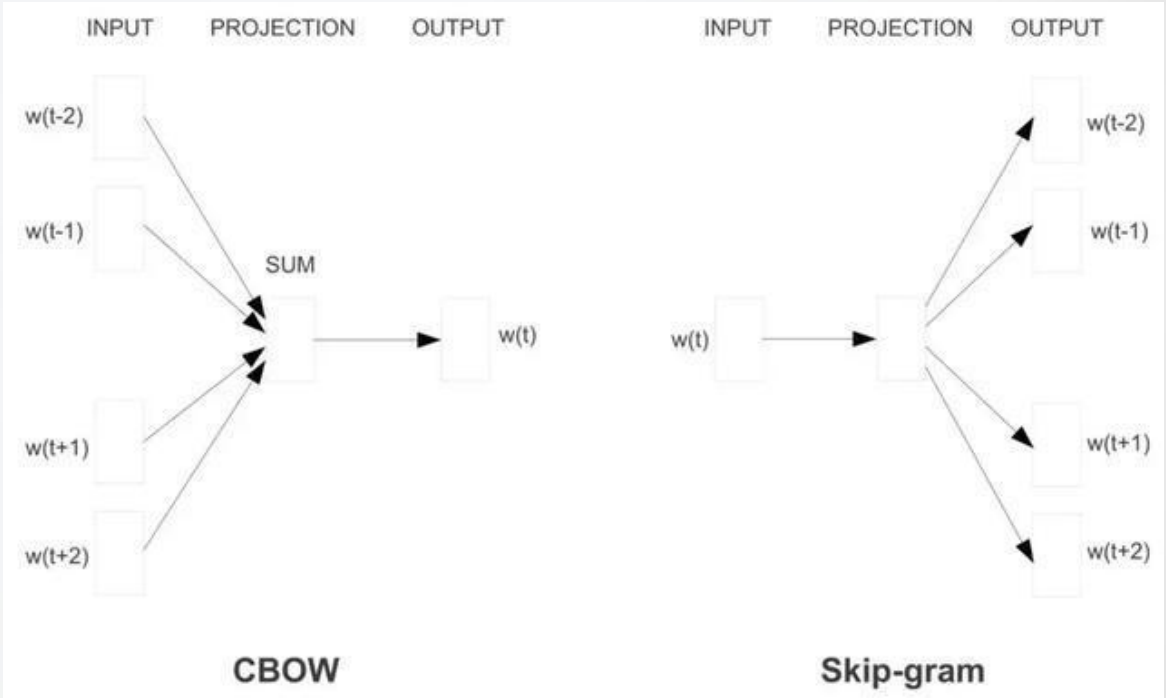
词嵌入：word2vec



1.4 发展历史

词嵌入：word2vec

Word2vec 有两种不同的实现方法：**CBOW**（Continuous Bag-of-Words）和 **Skip-gram**。它们在预测目标上有所不同：一个是根据周围的词语预测中心词语，另一个则恰恰相反。



1.4 发展历史

对比维度	早期神经语言模型（NNLM/ELMo前身）	Word2vec（CBOW/Skip-gram）
核心目标	精准预测文本序列，词向量为训练副产品	高效学习词的语义表征，词向量为核心目标
模型结构	深层网络（全连接/RNN），结构复杂	浅层网络（输入+隐藏+输出），极简结构
训练计算核心	传统全量softmax，无优化	负采样/层次Softmax，大幅降低计算量
计算复杂度	O(V)（V为词汇量），指数级增长	O(logV)，计算量降至原1/100以上
算力要求	极高，依赖专业高性能算力	极低，普通服务器/单机即可训练
可处理数据规模	小数据集（万级文本），无法适配大规模语料	大规模数据集（千万/亿级文本），兼容互联网语料
训练效率	极低，训练周期以周/月计	极高，训练周期以小时/天计
词向量泛化能力	弱，仅适配训练语料的小范围场景	强，具备语义相似性，可跨任务迁移
落地实用性	差，仅停留在实验室研究阶段	优，成为NLP通用工程化工具，快速落地各类任务
核心优势	序列预测精度高，语言建模能力强	训练快、成本低、词向量表征效果好，适配量产
核心短板	效率低、算力成本高、无法规模化应用	序列预测能力弱，仅聚焦词级语义表征

1.4 发展历史

NNLM（神经语言模型）序列预测精度高，具体原因有3点：

- 1. 模型结构适配序列，精准捕捉上下文关联
  - NNLM融合全连接层+循环结构（隐层），输入不是孤立词，而是连续的上下文词序列，逐层学习“前文词与待预测词”的语义，贴合文本的序列特性；
  - 而Word2vec为了效率，仅用浅层网络捕捉局部小窗口关联，丢失了长距离序列信息。
- 2. 全量softmax计算，输出精准的词汇表概率分布
  - NNLM用传统全量softmax对整个词汇表做归一化，直接输出每个词作为下一个词的精准概率，预测时会选择概率最高的词，从数学上保证了序列预测的准确性；
  - 反观Word2vec用负采样/层次Softmax近似替代全量softmax，牺牲了概率计算的精准度，只为快速学词向量。

```
【输入层】
连续上下文词序列（如：我/爱/自然/语言→预测下一个词）
        ↓
将每个词转为**固定维度词向量**，拼接成上下文向量矩阵
        ↓
【隐藏层（含循环结构）】
全连接+循环特征提取 → 捕捉上下文**长距离语义/语序关联**
        ↓
得到融合全部上下文信息的**整体特征向量**
        ↓
【输出层（全量Softmax）】
特征向量映射到**整个词汇表维度**
        ↓
全量Softmax归一化 → 输出**词汇表中每个词的预测概率**
        ↓
【预测结果】
选择概率最高的词 → 作为序列下一个词的精准预测
```

1.4 发展历史

NNLM（神经语言模型）序列预测精度高，具体原因有3点：

3. 聚焦单一目标，无训练目标的折中与妥协

NNLM的核心目标就是序列预测，所有模型设计、计算资源都围绕“精准预测下一个词”展开，无其他目标的干扰；

而Word2vec的核心目标是高效学习词向量，序列预测只是“辅助任务”，为了提升训练效率，主动放弃了序列预测的精度，属于“目标取舍”的结果。

总之：NNLM是“为序列预测而生”的模型，用适配的结构+精准的计算+单一的目标，实现了高序列预测精度；但也正因这些设计，带来了计算复杂、效率低的问题，而Word2vec正是通过舍弃这些“精准设计”，换来了训练效率的大幅提升。

1.4 发展历史

TIPS：为什么从神经语言模型发展到WORD2VEC?

- **核心原因：早期神经语言模型虽好，但训练速度极慢、算力成本高，无法处理大规模文本数据，而Word2vec通过简化模型结构，实现了高效的词向量训练，成为NLP领域的实用化突破。**
- **早期神经语言模型的核心痛点**
  - 1. 模型复杂，训练效率低：基于全连接/循环神经网络，需对整个词汇表做softmax计算，词汇量上万时计算量呈指数级增长。
  - 2. 依赖海量算力：仅能处理小数据集，无法利用互联网规模的文本（如维基百科、网页语料），学出的词向量泛化能力弱。
  - 3. 实用性差：训练周期长、资源消耗大，难以落地实际NLP任务。

1.4 发展历史

TIPS：为什么从神经语言模型发展到WORD2VEC?

- **Word2vec的核心改进（针对性解决痛点）**
  - 1. 简化模型结构：抛弃复杂的深层网络，用浅层神经网络（输入层+隐藏层+输出层）实现，大幅降低计算复杂度。
  - 2. 优化训练目标：提出CBOW（连续词袋）和SKIP-GRAM（跳字）两种轻量目标，不直接预测整句，而是通过“上下文预测中心词”或“中心词预测上下文”训练，聚焦词与词的关联。
  - 3. 创新计算方法：引入负采样（NEGATIVE SAMPLING）和层次SOFTMAX（HIERARCHICAL SOFTMAX），替代传统全量SOFTMAX，将计算量从O(V)（V为词汇量）降至O(LOGV)。
  - 4. 适配大规模数据：能高效处理千万级甚至亿级文本

1.4 发展历史

TIPS：为什么从神经语言模型发展到WORD2VEC?

- 核心发展逻辑：从“精准预测”到“高效表征”
  - 早期神经语言模型的核心目标是精准预测文本序列，词向量只是训练的“副产品”；
  - WORD2VEC彻底转变思路，将词向量的高效学习作为核心目标，放弃精准的序列预测，通过“轻量任务+高效计算”让词向量成为NLP的通用基础工具，实现了从“研究模型”到“工程化工具”的跨越。
- WORD2VEC不是否定神经语言模型，而是对其做了极致的工程化简化，在保证词向量表征效果的前提下，解决了训练效率和算力的核心问题，让词向量从实验室走向实际应用，成为后续NLP（如BERT、GPT）的重要基础。

1.4 发展历史

TIPS：过渡

- 随着WORD2VEC的提出，在2013到2014年，各种神经网络模型在NLP上的应用也逐渐增多，其中应用最广泛的是：卷积神经网络CNN、循环神经网络RNN、结构递归神经网络（RECURSIVE NEURAL NETWORKS）。

1.4 发展历史

自然语言处理发展复苏期的8个里程碑

4、2013年——用于NLP的神经网络（Neural networks for NLP）

- 2013年和2014年是NLP领域神经网络时代的开始。三种类型的神经网络使用最为广泛：**循环神经网络**、**卷积神经网络**和**结构递归神经网络**。
- **RNN**是 NLP 领域处理动态输入序列最自然的选择，之后很快被长短期记忆网络LSTM代替，LSTM被用于同时处理出现在左侧和右侧的文本内容。
- **CNN**的一个优点是具有更好的并行性，因为卷积操作中每个时间步的状态只依赖于局部上下文，而不是RNN中依赖于所有过去的状态。
- **结构递归神经网络：**从语言学的角度来看，语言是具有层级结构的：词语组成高阶的短语和小句，它们本身可以根据一定的规则**递归地组合**。

1.4 发展历史

CNN

- 在CNN出现以前，对于图像的处理一直都是一个很大的问题。
- 一张512*521的灰度图，它的输入参数就达到252144个，更别提1024*1024*3的彩色图
- CNN将大量的参数降维成少量的参数



1024*1024*3　　100*100*3

1.4 发展历史——CNN和RNN

01. 电脑如何“看”图片?

02. CNN核心思想：分层识别

03. 卷积层：提取图像特征

04. 池化层：压缩与降维

05. 全连接层：最终分类决策

06. CNN完整工作流程

07. 总结与应用场景

1.4 发展历史——CNN和RNN

电脑如何“看”图片？



人类视角

直观识别耳朵、眼睛、鼻子等生物特征。

电脑视角：数字矩阵

3	5	6	7	3	4	5	6	7
8	7	10	7	18	8	9	9	1
17	13	22	15	17	8	9	19	10
10	21	14	21	15	16	7	9	9
4	11	22	15	26	7	8	35	7

电脑“看”到的不是图像，而是由数字组成的矩阵。每个像素点都由一组数字（如RGB值）来精确表示颜色和亮度。

核心挑战：从数字到语义

电脑本身无法直接理解这些数字的含义。它需要通过特定的深度学习算法（如CNN）来自动提取图像中的关键特征，从而完成从“数字矩阵”到“这是一只猫”的认知过程。

1.4 发展历史——CNN和RNN

CNN的核心思想：分层识别

第一层：低层特征提取

识别图像中的简单特征，如边缘、线条、明暗变化等基础视觉信号。

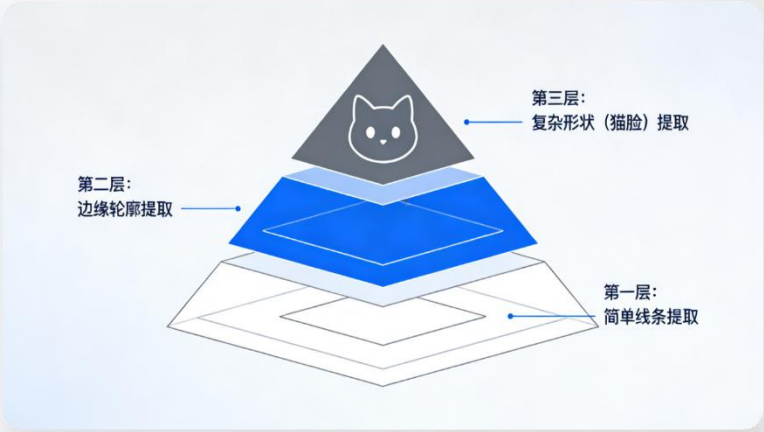
第二层：中层特征组合

将简单特征组合，形成更复杂的形状，如眼睛、耳朵、轮子等部件。

第三层：高层特征识别

进一步组合中层特征，识别出完整的物体，如猫脸、汽车、人脸等。

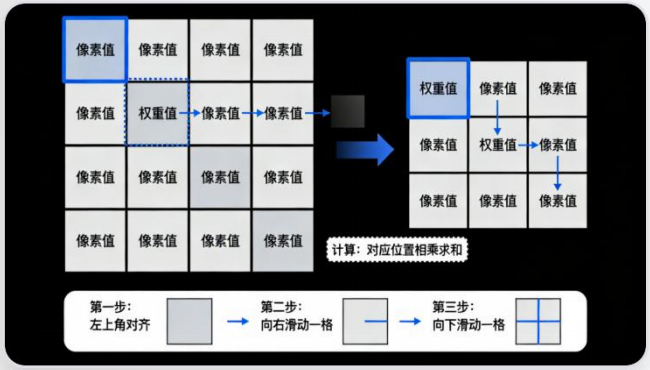
核心逻辑：从简单到复杂、从局部到整体，逐层抽象，高效理解图像内容。



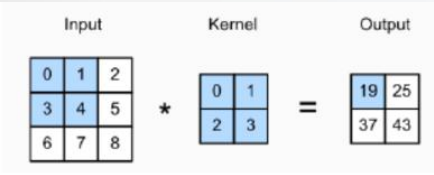
CNN特征提取金字塔模型示意图

1.4 发展历史——CNN和RNN

卷积层：提取图像特征



图：卷积核在像素矩阵上的滑动计算过程



卷积核（过滤器）

类似“小放大镜”的小型矩阵，充当特征探测器，负责识别特定模式。

滑动计算机制

卷积核在图像上从左到右、从上到下滑动，通过加权求和提取区域特征（如边缘、线条）。

生成特征图

每个卷积核生成一张特征图，突出显示图像中符合该核检测特征的区域。

0*0+1*1+3*2+4*3=19
卷积核的高度和宽度都是2

核心价值：通过多层卷积与池化，将原始像素数据转化为高层语义特征，实现从“看得到”到“看得懂”。

1.4 发展历史——CNN和RNN

池化层：压缩与降维

核心作用：数据压缩与特征保留

在减少数据维度的同时，保留关键特征信息，去除冗余。

下采样 (Downsampling)

对卷积层输出的特征图进行尺寸缩小，大幅降低后续计算量。

最大池化 (Max Pooling)

在小区域（如2x2）内取最大值作为代表，保留最显著特征。

关键优势

防止过拟合，增强模型对图像平移、旋转等变形的鲁棒性。



最大池化操作示意图 (2x2 Kernel)

工作原理总结：

池化层像一个“过滤器”，通过在局部感受野内取最大值，将4x4的特征图“压缩”为2x2，同时保留了数值最大的、最具辨识度的特征信息。

1.4 发展历史——CNN和RNN

全连接层：最终的分类决策

核心作用：特征整合与决策

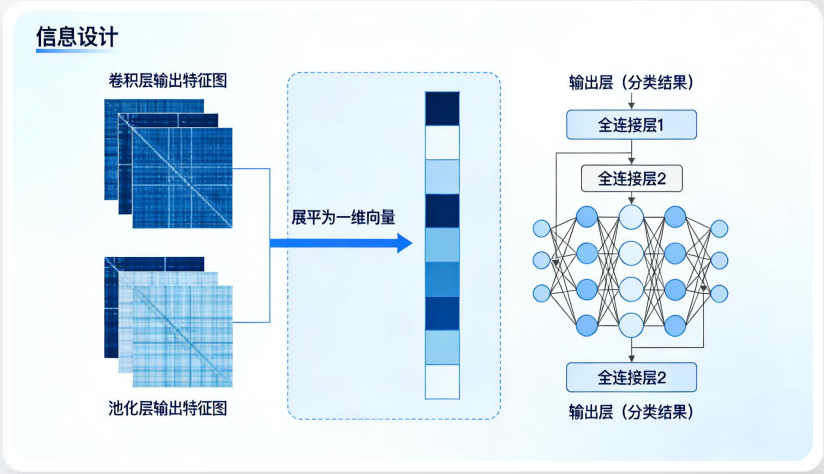
在多层卷积池化后，全连接层整合高级特征并输出分类概率，完成最终决策。

关键步骤：特征展平 (Flatten)

将多维的特征图压缩展平为一维向量，为全连接计算做好数据格式准备。

分类决策：全连接计算

神经元全连接计算权重，整合特征信息，输出各类别的概率分布结果。



全连接层工作原理示意图：从特征图到分类输出

总结：全连接层（Fully Connected Layer）是CNN的“大脑”，它通过将空间特征转化为全局特征，实现从特征提取到语义分类的跨越。

1.4 发展历史——CNN和RNN

CNN完整工作流程

1. 输入层
输入原始图像（像素矩阵）作为网络起点。

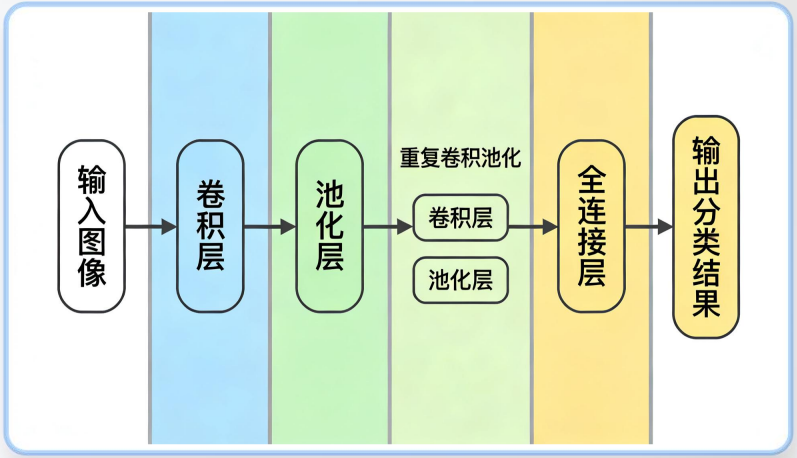
2. 卷积层
利用卷积核提取边缘、线条等低级特征。

3. 池化层
对特征图降维，保留关键信息并减少计算。

4. 重复堆叠
多层网络逐步提取形状、物体等高级特征。

5. 全连接层
整合所有特征，进行最终的分类或回归计算。

6. 输出结果
输出图像的分类标签（如猫/狗）或预测值。



CNN 网络结构与数据流示意图

核心逻辑：CNN通过“局部感知”和“权值共享”大幅降低参数量，利用卷积提取特征、池化压缩信息，最终通过全连接层实现从“像素”到“语义”的理解与分类。

1.4 发展历史——CNN和RNN

总结与应用

CNN 核心优势

分层特征提取
从简单到复杂，模拟人类视觉机制，自动提取图像特征。

权值共享机制
大幅减少参数数量，降低计算复杂度，有效防止过拟合。

平移不变性
对图像中物体的位置变化不敏感，具备极强的鲁棒性。

广泛应用场景

图像识别： 人脸识别、物体识别、验证码识别等。

目标检测： 在复杂图像中精准定位并识别多个物体。

自动驾驶： 识别道路、车辆、行人，辅助车辆决策。

图像分割： 像素级分割图像对象，用于精细场景分析。

医疗影像： 辅助X光片、CT扫描等医学影像的诊断。

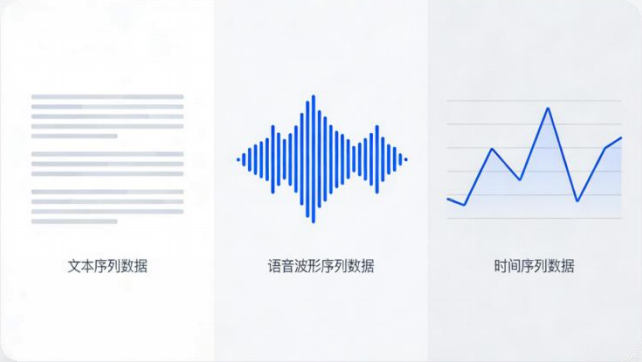
1.4 发展历史——CNN和RNN

- 什么是序列数据?
  定义与特征解析
- RNN的核心思想
  处理序列的“记忆”网络
- RNN的工作原理
  循环机制与信息传递
- RNN的改进模型
  LSTM与GRU详解
- RNN的应用场景
  语音、文本与时间序列
- RNN vs CNN
  核心区别与适用场景
- 总结与展望
  未来发展趋势

1.4 发展历史——CNN和RNN

什么是序列数据?

定义：序列数据是指数据之间存在顺序关系，当前数据的含义依赖于前面的数据，具有强时序关联性。



文本数据

单词顺序决定语义，如句子

语音数据

声音信号随时间连续变化

时间序列数据

按时间轴排列，如股价、气温

核心挑战：传统神经网络（如CNN）难以捕捉这种长距离的时序依赖关系。

1.4 发展历史——CNN和RNN

RNN的核心思想：处理序列的“记忆”网络



循环连接机制

隐藏层输出不仅作为当前输出，还会反馈至自身，作为下一时刻的输入。

隐藏状态（Hidden State）

反馈的信息即“记忆”，携带了之前时间步的历史信息，影响当前计算。

序列数据处理

结合历史“记忆”与当前输入，赋予网络理解上下文和时序依赖的能力。

Recurrent Neural Network: Memory for Sequences

1.4 发展历史——CNN和RNN

RNN的工作原理：循环与信息传递

时间步 t=1：初始输入

输入首个单词，结合初始隐藏状态（通常为0），计算首个隐藏状态与输出。

时间步 t=2：信息传递

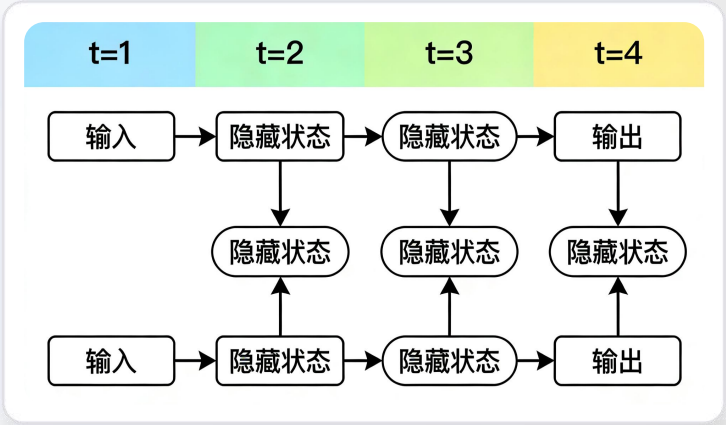
输入第二个单词，结合上一步隐藏状态，迭代更新新的状态与输出。

后续时间步：循环累积

重复上述过程，每一步的隐藏状态持续累积包含之前所有输入的历史信息。

最终输出：任务导向

根据具体任务需求，输出可以是最后一个时间步的结果，也可以是全序列输出。



RNN 时间步展开示意图

1.4 发展历史——CNN和RNN

RNN的改进：LSTM与GRU

RNN的瓶颈：梯度消失

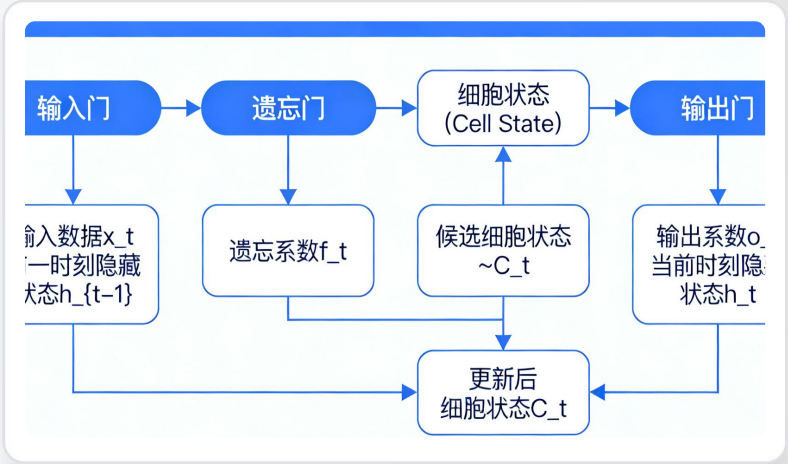
处理长序列时难以记住长期依赖关系，信息传递受阻。

LSTM：长短期记忆网络 (门控机制)

- **遗忘门：**决定从细胞状态中丢弃哪些历史信息。
- **输入门：**决定哪些新信息被存储到细胞状态中。
- **输出门：**决定从细胞状态中输出哪些信息作为结果。
- **细胞状态：**像传送带一样，信息线性交互，易于保持。

GRU：门控循环单元 (简化版)

将遗忘门和输入门合并为“更新门”，结构更简洁，计算效率更高。



图：LSTM 内部门控结构示意图

1.4 发展历史——CNN和RNN

RNN的应用场景



自然语言处理

机器翻译、文本生成、情感分析、聊天机器人。



语音识别

将语音信号转换为文字，如语音输入。



时间序列预测

股票价格预测、天气预报、交通流量预测。



视频分析

视频内容理解、动作识别与行为分析。

1.4 发展历史——CNN和RNN

RNN vs CNN：核心区别与适用场景

对比维度	RNN (循环神经网络)	CNN (卷积神经网络)
核心能力	处理序列数据，具有记忆能力	处理网格状数据（如图像），特征提取
网络结构	循环连接，信息在时间步间传递	卷积层和池化层，权值共享机制
数据类型	序列数据（文本、语音、时间序列）	网格数据（图像、视频帧、像素矩阵）
关键特性	时间依赖性、短期/长期记忆性	平移不变性、局部感受野、特征层级
典型应用	机器翻译、语音识别、股价预测	图像分类、目标检测、人脸识别

总结：RNN 擅长处理有序的、前后依赖的序列数据；CNN 擅长处理有空间结构的网格数据。

1.4 发展历史——CNN和RNN

总结与展望

核心回顾：RNN 关键特性

- **序列处理与记忆：**
  专为序列数据设计，通过循环结构赋予网络“记忆”能力，捕捉上下文信息。
- **改进模型的突破：**
  LSTM和GRU等变体有效解决了长期依赖问题，大幅提升了模型在长序列任务中的表现。
- **与CNN的互补：**
  RNN侧重序列时序，CNN侧重网格空间，二者构成深度学习处理不同数据类型的基石。

未来展望：演进与融合

- **向 Transformer 演进：**
  虽然Transformer成为NLP主流，但其核心思想（如注意力机制）仍受RNN序列建模思想的启发与影响。
- **持续的生命力：**
  在语音识别、时间序列预测等特定领域，RNN及其变体（如GRU）因其高效性仍占据重要地位。
- **模型融合趋势：**
  未来模型将更倾向于结合CNN的特征提取能力与RNN/Transformer的序列建模能力。

“RNN 不仅是深度学习历史上的里程碑，更是理解序列数据处理逻辑的关键钥匙。”

自然语言处理发展复苏期的8个里程碑

5、2014年——序列到序列模型（Sequence-to-sequence models）

- 2014 年，Sutskever 等人提出了**序列到序列学习**，即使用神经网络将一个序列映射到另一个序列的一般化框架。
- 在这个框架中，一个作为**编码器**的神经网络对句子符号进行处理，并将其压缩成向量表示；
- 然后，一个作为**解码器**的神经网络根据编码器的状态逐个预测输出符号，并将前一个预测得到的输出符号作为预测下一个输出符号的输入。

1.4 发展历史——序列到序列模型

生活中的序列数据



文本序列

一句话、一篇文章



音符序列

一段音乐、一首歌



时间序列

股价走势、天气变化



声波序列

一段语音、一首歌



共同特点：这些数据都是**按顺序排列**的，前后元素之间存在天然的依赖关系。

1.4 发展历史——序列到序列模型

什么是序列到序列模型?



核心定义：输入序列 ➔ 输出序列

序列到序列（Seq2Seq）模型是一种处理序列数据的深度学习架构。它的核心能力是：

- 理解一种序列的含义
- 将其转换成另一种序列

你可以把它想象成一位“智能翻译官”，在不同的语言、格式或模态之间建立精准的桥梁。

1.4 发展历史

序列到序列模型

- **1. 编码器：** 把输入的任意长度序列（如中文句子），压缩成固定维度的上下文向量，只保留核心语义，丢掉冗余信息（像把一篇文章总结成一个核心关键词包）；
- **2. 上下文向量：** 是输入序列的“语义精华”；
- **3. 解码器：** 从上下文向量中，逐词生成任意长度的输出序列（如英文句子），生成时会结合已生成的词，保证语序和语义通顺。

1.4 发展历史——序列到序列模型

序列到序列模型

□**核心定义：** 专门解决“一个序列输入→另一个序列输出”的NLP模型，输入和输出序列的长度、内容可自由匹配，是处理翻译、摘要、对话等非固定长度序列任务的核心模型。

□**举例：** 把“一句话/一段文字”变成“另一句话/一段文字”，比如中译英、长文缩短句、问答对话，都是Seq2Seq的典型场景。

□**工作流程：** 【输入序列】→ 编码器（Encoder）→ 上下文向量（核心语义）→ 解码器（Decoder）→ 【输出序列】

1.4 发展历史——序列到序列模型

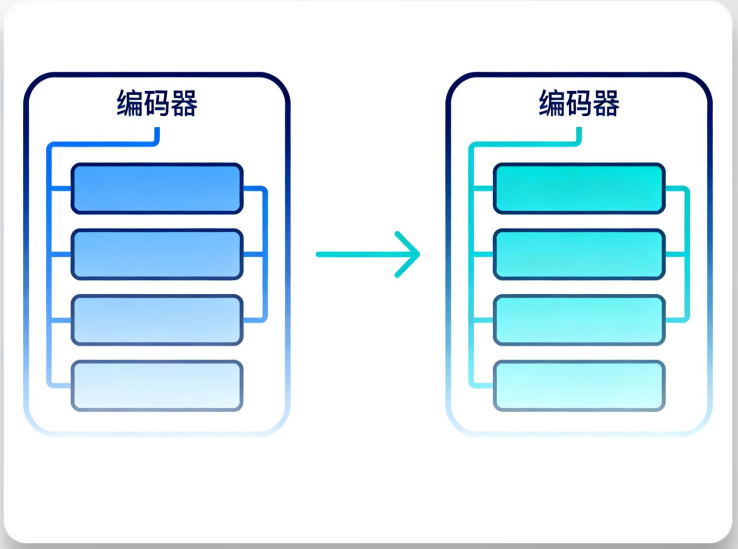
核心架构：编码器-解码器

编码器 (Encoder)

负责“理解”输入序列，将原始信息转化为机器可识别的向量表示。

解码器 (Decoder)

负责“生成”输出序列，基于编码器的理解结果，逐步构建目标响应。



“理解”与“表达”的完美闭环，构成了 Seq2Seq 模型的核心智能。

1.4 发展历史——序列到序列模型

编码器：理解输入

编码器的核心作用是“阅读”并“理解”输入序列。它像人类大脑一样，逐字逐句分析信息，最终形成对整体的认知。

信息压缩与总结
将长序列信息提炼为核心要素

生成语义向量 (Context Vector)
固定长度向量，包含输入序列的核心含义



编码器如同大脑处理信息

这个向量是解码器进行“翻译”或“生成”的基础，它承载了模型对输入的全部理解。

1.4 发展历史——序列到序列模型

解码器：生成输出

核心机制：序列生成

接收编码器生成的“语义向量”，通过循环网络一步步解码，最终生成目标语言或文本序列。

形象比喻：智能作家

如同作家获取核心思想后，不会凭空捏造，而是逐字逐句地创作出逻辑连贯、内容完整的文章。



信息处理与生成过程可视化

1.4 发展历史——序列到序列模型

第一步：编码阶段



输入序列

例如：“我爱中国”



编码器处理

神经网络深度分析



输出语义向量

包含核心信息的向量

在编码阶段，输入序列（例如“我爱中国”）被送入编码器。编码器通过内部的神经网络处理，逐字分析并理解其含义，最终输出一个包含了输入序列核心信息的语义向量。

1.4 发展历史——序列到序列模型

第二步：解码阶段

接收语义向量

解码器接收来自编码器生成的高维语义向量，作为翻译的基础输入。

逐词生成文本

基于向量信息，模型逐个预测并生成目标语言的单词，构建翻译序列。

输出完整序列

将生成的单词按顺序组合，形成符合语法和语义的完整目标文本。

翻译示例：“我爱中国” → “I love China”

输入向量 ➔ 生成单词 “I”

上下文 + 向量 ➔ 生成单词 “love”

上下文 + 向量 ➔ 生成单词 “China”

1.4 发展历史——序列到序列模型

关键技术：注意力机制



机制引入：为了解决Seq2Seq模型在长序列生成中信息丢失的问题，引入了“注意力机制”。它赋予了解码器“聚焦”能力，使其在生成每个词时，能主动关注输入序列中最相关的信息。



直观类比：这就像人类进行翻译工作时，并非一次性读完所有原文，而是在书写译文的每一个词时，都会将目光重新聚焦到原文中对应的那个或那些关键词上，从而确保翻译的准确性和流畅性。

核心价值：提升长文本生成的准确性与连贯性，让AI更“聪明”地理解上下文

1.4 发展历史——序列到序列模型

序列到序列模型

- 编码器将输入文本转换为表示向量，解码器将向量转化为输出文本序列。这个结构一经提出便迅速被应用到机器翻译，并逐步取代了基于短语的整句机器翻译模型。
- 但这种模型存在一些**缺点**：
  - 1、编码器需要将整个序列信息压缩进一个固定长度语义向量c里，会产生**信息丢失**，语义向量c无法完整表达整个序列的信息；
  - 2、先输入的内容携带的信息，会被后输入的稀释掉或覆盖掉，序列越长这样的现象越严重。c的长度成了限制模型**性能的瓶颈**。

1.4 发展历史——序列到序列模型

序列到序列模型

- 应用场景：
  - - 机器翻译：输入「我喜欢自然语言处理」→ 输出「I like natural language processing」
  - - 文本摘要：输入长新闻→输出短摘要
  - - 智能对话：输入「今天天气怎么样？」→输出「今天晴，气温20℃」
  - - 文本生成：输入「以春天为主题写一句话」→输出「春风拂面，花开遍野」

1.4 发展历史

自然语言处理发展复苏期的8个里程碑

□6、2015年——注意力机制

- 注意力机制是神经网络机器翻译 (NMT) 的核心创新之一，也是使神经网络机器翻译优于经典的基于短语的机器翻译的关键。
- **序列到序列**学习的主要**瓶颈**是，需要将源序列的全部内容压缩为固定大小的向量。**注意力机制**通过让解码器回顾源序列的隐藏状态，以此为解码器提供加权平均值的输入来**缓解这一问题**。

1.4 发展历史——注意力机制

- 为了弥补这种模型的局限性，在2015年Dzmitry Bahdanau等人提出了注意力机制（Attention）。
- 注意力机制作为一种资源分配方案，对不同的内容分配不同的注意力权重，
- 将有限的计算资源用来处理更重要的信息，是解决信息超载问题的主要手段。

1.4 发展历史——注意力机制

- 注意力机制可以视为**模糊记忆**的一种形式，
- 其记忆的内容包括模型之前的隐藏状态，由模型选择从记忆中检索哪些内容。
- 与此同时，更多**具有明确记忆单元**的模型被提出。

1.4 发展历史——注意力机制

传统AI的困境：“雨露均沾”的低效处理



处理方式“一视同仁”

传统模型（如RNN）对所有输入信息给予同等关注，缺乏优先级判断。

难以区分信息权重

面对长文本或复杂图像时，无法识别关键特征，导致“眉毛胡子一把抓”。

信息过载与效率损耗

无效信息占用大量算力，不仅处理速度慢，还容易丢失核心细节。

类比：手电筒式的视野

如同在黑暗中用手电筒看书，只能聚焦局部，无法高效把握全局信息。

1.4 发展历史——注意力机制

注意力机制：让AI学会“抓重点”



核心思想

摒弃平均主义，根据任务目标动态计算信息重要性权重。

聚焦重点

识别关键信息并分配高权重，集中资源进行深度处理。

忽略次要

对无关或次要信息分配低权重，减少无效干扰。

形象比喻：这就像一个智能聚光灯，能够根据需要，精准照亮信息中的重点部分，让AI处理更高效。

1.4 发展历史——注意力机制

解密注意力机制的核心原理QKV：查字典的故事



Query (Q) - 查询

你要查的“问题”或“需求”
例如：“什么是苹果？”



Key (K) - 键

字典里的“索引”或“关键词”
例如：所有包含“苹果”的词条



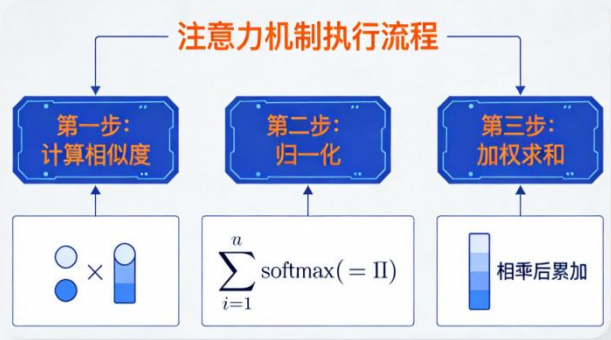
Value (V) - 值

词条对应的“解释”或“内容”
例如：“苹果是一种水果...”

核心逻辑：注意力机制通过Query匹配Key，再根据匹配度提取对应的Value，从而找到最相关信息

1.4 发展历史——注意力机制

三步工作流程：计算、归一化、加权



01 计算相似度

用Query去匹配所有Key，计算它们之间的相关性得分。得分越高，说明两者越相关。

02 归一化处理

将所有得分转化为权重，确保所有权重之和为1。这样能直观判断每个Key的重要程度。

03 加权求和

利用权重对对应Value进行加权求和，得到最终输出结果，该结果聚焦了核心关注信息。

1.4 发展历史——注意力机制

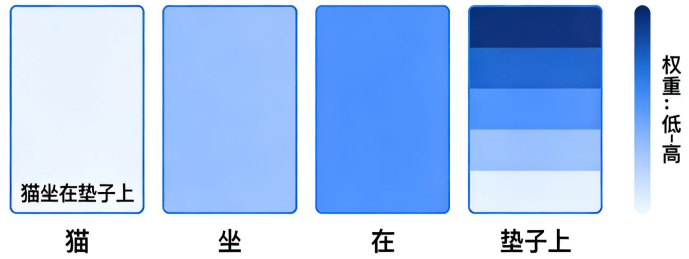
自注意力：序列内部的“对话”

核心机制：Query 与 Key 的交互

在处理序列时，每个元素作为Query，与序列中所有其他元素（Key）计算相似度。这种机制让模型能自动捕捉序列内部的依赖关系。

场景示例：理解语义关联

以句子“猫坐在垫子上”为例，模型处理“坐”时，会分配更高的注意力权重给“猫”和“垫子”，从而精准理解动作的执行者和承受者。

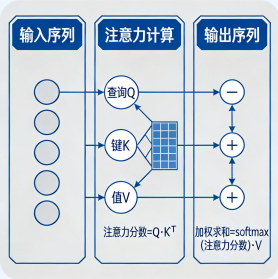


句子“猫坐在垫子上”的注意力权重分布热力图

核心价值：打破距离限制，让模型具备全局视野与上下文感知能力

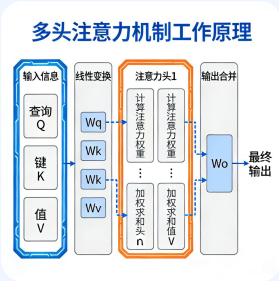
1.4 发展历史——注意力机制

注意力机制的“家族成员”



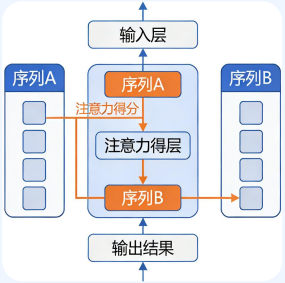
自注意力 (Self-Attention)

序列内部元素相互关注，捕捉内部依赖关系，是Transformer架构的核心基石。



多头注意力 (Multi-Head)

同时运行多个注意力机制，每个“头”关注不同类型的信息，整合结果以捕捉更丰富的特征。



交叉注意力 (Cross-Attention)

用于不同序列间的关注。例如文本序列作为Query，图像序列作为Key/Value，实现跨模态交互。

1.4 发展历史——注意力机制

NLP应用：精准的语言翻译官



注意力机制 (Attention Mechanism)

在生成目标语言的每个词时，模型能动态关注源语言中“最相关”的词汇，解决长句信息丢失问题。

精准对齐示例

翻译“Cat sits on the mat”时，生成“猫”关注“Cat”，生成“垫子”聚焦“mat”，实现词级精准映射。

核心价值与成果

这种精准的对齐能力，打破了传统翻译瓶颈，显著提升了译文的准确性与自然流畅度。

1.4 发展历史——注意力机制

CV应用：精准的图像识别专家



输入图像：猫与狗

聚焦关键区域

在图像分类任务中，注意力机制能够帮助模型自动忽略无关背景，将计算资源和关注点聚焦于图像中的核心特征区域。

智能对象识别

例如在识别“猫和狗”时，模型会像人类视觉一样，精准锁定动物主体，而非周围的空白或干扰元素。

增强可解释性

通过可视化的热力图，我们可以直观地看到模型关注的“热点”，这不仅验证了模型的有效性，也极大提升了AI决策的透明度。

1.4 发展历史——注意力机制

CV应用：创意的图像生成大师



AI生成示例：戴墨镜的猫

注意力机制：理解与映射

在文本生成图像任务中，模型通过注意力机制解析文本语义，并将其精准映射到图像的不同区域，确保生成内容的逻辑性。

精准捕捉：关键词驱动细节

例如输入“一只戴着墨镜的猫”，模型会重点关注“猫”的主体特征与“墨镜”的配饰细节，从而生成高度还原描述的图像。

创意赋能：从文本到艺术

这种能力让AI突破了简单的图像拼凑，能够根据抽象的文字描述，自主创作富有创意、细节丰富且符合逻辑的视觉作品。

1.4 发展历史——注意力机制

总结：注意力机制的核心价值

提升效率

让AI从“粗放式”处理转向“精细化”理解，更高效地利用计算资源。

增强性能

显著提升NLP、CV等领域模型性能，带来了质的飞跃。

推动革命

Transformer的核心基石，开启AI新篇章，奠定通用人工智能基础。

1.4 发展历史

自然语言处理发展复苏期的8个里程碑

7、2015年——基于记忆的神经网络

- 注意力机制可以视为模糊记忆的一种形式，其记忆的内容包括模型之前的隐藏状态，**由模型选择从记忆中检索哪些内容**。
- 与此同时，更多具有明确记忆单元的模型被提出。
- 他们有很多不同的变化形式，比如记忆网络（Memory Network）、端到端的记忆网络（End-to-end Memory Newtorks）、动态记忆网络（DynamicMemory Networks）。

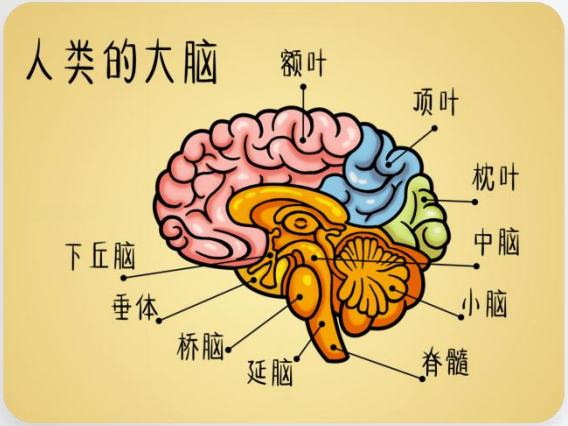
1.4 发展历史—基于记忆的神经网络



基于记忆的神经网络：让AI拥有“记忆”的魔法

1.4 发展历史—基于记忆的神经网络

人类的记忆：大脑的“数据库”



强大的记忆系统

人类大脑如同高效数据库，具备海量信息的存储与快速检索能力。

双重记忆机制

短期记忆处理速度快但容量有限；长期记忆容量巨大，负责信息的永久保存。

学习与应用闭环

通过持续学习丰富长期记忆库，并在解决问题时快速检索调用相关知识。

1.4 发展历史—基于记忆的神经网络

传统神经网络的困境：“健忘”的AI



核心缺陷：短期记忆限制

传统CNN和普通RNN在处理信息时存在“健忘”问题，缺乏长期记忆能力，难以维持信息的连贯性。

性能瓶颈：信息丢失

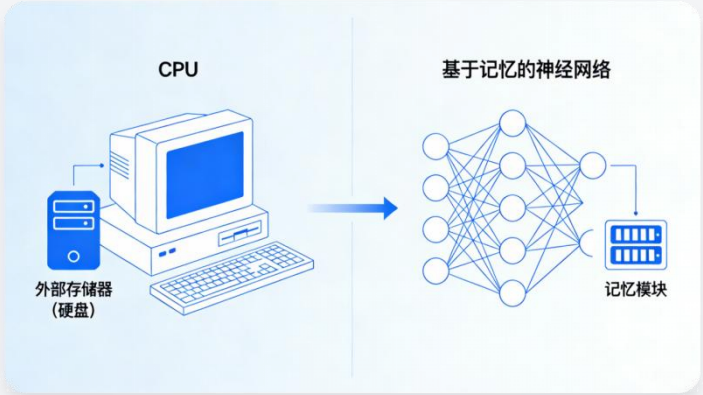
面对长序列或复杂任务时，早期输入的关键信息会被逐渐覆盖和遗忘，导致模型无法建立长距离依赖。

形象类比：没有笔记本的学生

如同学生只能依赖大脑的短期记忆，无法记录和翻阅笔记，导致无法回顾和利用之前学过的重要知识点。

1.4 发展历史—基于记忆的神经网络

基于记忆的神经网络：给AI装上“记忆硬盘”



核心思想

给AI装上一个“记忆硬盘”，通过外部存储突破传统网络的记忆限制。

技术融合

将神经网络强大的模式识别能力与可读写的外部存储器相结合。

能力提升

像人类一样存储重要信息，在需要时快速检索，拥有真正的长期记忆。

1.4 发展历史—基于记忆的神经网络

两大核心组件：控制器与外部存储器



控制器 (Controller)

就像AI的“大脑”，通常是一个神经网络（如LSTM或MLP）。它负责接收输入数据，处理信息，并决定如何与外部存储器进行交互（读或写）。



外部存储器 (External Memory)

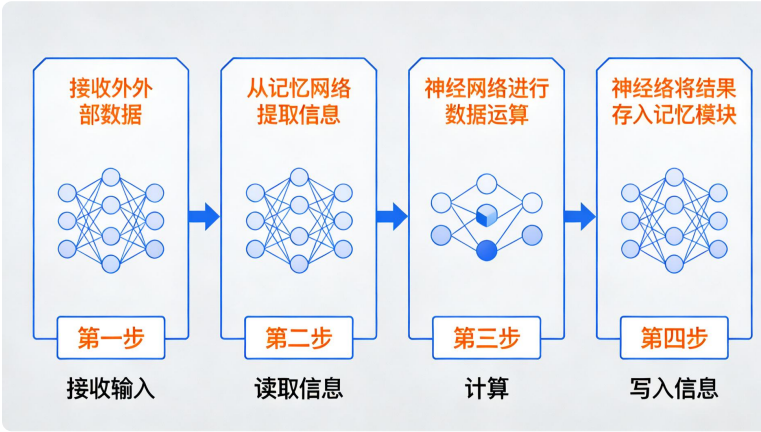
就像AI的“笔记本”或“数据库”，是一个大容量的存储单元，可以存储大量的信息。它允许控制器进行快速的读写操作。

协同工作：控制器负责“思考决策”，外部存储器负责“记忆存储”，共构成具备记忆能力的系统

1.4 发展历史—基于记忆的神经网络

工作流程：智能的读写操作

- 接收输入：控制器接收外部输入数据
- 决定读取：分析输入，确定需读取的信息
- 读取信息：从存储器中提取相关信息
- 计算与决策：结合数据计算，得出结果
- 决定写入：判断是否需写入新信息
- 写入信息：将新信息写入，更新记忆



图示：智能读写操作的核心循环流程

核心逻辑：感知 → 检索 → 计算 → 存储

1.4 发展历史—基于记忆的神经网络

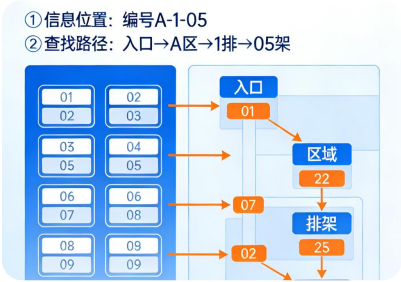
寻址机制：如何找到需要的记忆？

基于内容的寻址 (Content-based)



类比：根据书名在图书馆找书。
原理：控制器根据信息的内容特征（如关键词），在存储器中查找最相似的信息。

基于位置的寻址 (Location-based)



类比：根据书的编号在图书馆找书。
原理：控制器根据信息在存储器中的物理或逻辑位置进行查找，特别适合处理序列信息。

核心价值：两种寻址方式结合使用，赋予了AI系统高效检索和灵活关联记忆的能力

1.4 发展历史—基于记忆的神经网络

经典模型：神经图灵机（NTM）：AI界的“图灵机”



基于记忆的神经网络模型

由DeepMind在2014年提出，是首个结合外部记忆模块的神经网络架构。

模拟图灵机工作原理

包含“控制器”（神经网络主体）和“外部存储器”，通过读写头实现数据的长期存储与交互。

强大的推理与记忆能力

能够自主学习复制、排序等简单算法，突破了传统神经网络的记忆局限，展现逻辑推理潜力。

1.4 发展历史—基于记忆的神经网络

经典模型：记忆增强神经网络（MANN）：更强大的记忆能力



MANN 核心模块结构示意图

模型起源与演进

MANN 是在神经图灵机 (NTM) 基础上发展而来的进阶模型，继承了外部记忆存储的核心思想。

核心架构升级

通常采用 LSTM 等更强大的神经网络作为控制器，并引入了更灵活、高效的记忆寻址与读写机制。

卓越的任务表现

在小样本学习 (Few-shot Learning)、图像分类等任务中表现优异，具备快速学习新概念的能力。

核心价值：通过外部记忆扩展神经网络容量，突破传统模型的“记忆力”瓶颈

1.4 发展历史—基于记忆的神经网络

NLP应用：智能问答助手



广泛的应用场景

覆盖电商客服机器人、智能家居语音助手、企业内部知识库查询等领域，提供24小时即时响应。

外部记忆增强机制

模型将海量知识库存储在外部存储器中。当用户提问时，控制器分析语义，精准检索相关知识片段，并生成自然语言答案。

精准与深度的答案生成

突破传统规则限制，能够处理复杂的多轮对话和推理问题，提供更准确、丰富且上下文连贯的回复。

基于外部记忆的神经网络架构，让机器不仅“听懂”，更能“理解”与“记忆”

1.4 发展历史—基于记忆的神经网络

NLP应用：更精准的机器翻译

智能翻译演示

Hello, how are you?

你好，你好吗?

核心工作原理

模型将词汇与语法规则存储于外部存储器中。在翻译过程中，控制器动态参考这些知识，生成更准确的译文。

技术优势

突破传统模型限制，能够有效处理长句和复杂的语法结构，显著提高翻译的准确性与可读性。

应用场景：广泛应用于跨语言交流、文档自动翻译及全球化内容分发，解决语言障碍问题。

1.4 发展历史—基于记忆的神经网络

CV应用：小样本学习专家

Few-Shot Learning 示例



新类别：猫 (Cat)

仅需 2-3 张样本即可识别



新类别：狗 (Dog)

无需大规模重新训练



快速泛化能力

利用记忆库知识进行推理



工作原理：外部记忆增强

模型将已学习的知识存储在外部存储器中。遇到新任务时，控制器提取新样本特征，与记忆库中的知识进行比对和结合，从而快速学习新的概念，无需从零开始。



核心优势：低依赖与高泛化

大大降低了对海量标注数据的依赖，解决长尾分布问题。同时显著提高了模型的泛化能力，使其能够适应现实世界中不断出现的新类别和新场景。

“只需少量样本，即可像专家一样快速掌握新知识”

1.4 发展历史—基于记忆的神经网络

总结：基于记忆的神经网络的核心价值

突破记忆瓶颈

赋予AI长期记忆能力，彻底解决传统模型的“健忘”问题。

增强推理能力

通过知识存储与检索，支持复杂的逻辑推理与问答任务。

推动AI发展

为实现通用人工智能(AGI)奠定基础，向类人智能迈进。

核心价值点



突破记忆瓶颈　增强推理能力　推动AI发展

1.4 发展历史—基于记忆的神经网络

TIPS：过渡

- 尽管各类神经模型在NLP任务中取得了成功，但由于大多数监督NLP任务的当前**数据集非常小**，而且深度神经网络通常具有**大量的参数**，使得它们在这些小的训练数据上**过于拟合**，在实际应用中不能很好地推广。因此，预训练模型逐步开始被更多学者重视。

1.4 发展历史—基于记忆的神经网络

TIPS：过渡

- 大量的研究表明，在大型语料库上的预训练模型（PTMs）可以学习**通用语言表示**，这有利于后续的NLP任务，避免从头开始训练新模型。
- 因此，一系列预训练语言模型，如BERT等应运而生，并在各类自然语言处理领域的下游任务中取得了良好的效果。

1.4 发展历史—预训练语言模型

自然语言处理发展复苏期的8个里程碑

8、2018年——预训练语言模型

- **第一代预训练语言模型**学习是与上下文无关的、静态的词向量：把词汇表中的每一个词汇映射到一个lookup table中，训练过程就是得到这个lookup table的过程。得到这个lookup table后，把每个词的One-hot乘以lookup table就得到这个词的词向量了。但这个无法处理一词多义等语言现象，也无法处理Out of Vocabulary问题。
- **第二代预训练语言模型**学习是与上下文相关的、动态的词向量：重要代表是ELMo、OpenAI GPT 和BERT 。
- 目前，BERT已经刷新了各种NLP榜单，成果惊人。



预训练语言模型：AI语言能力的革命

—— AI的“语言预科班”

1.4 发展历史—预训练语言模型

传统语言模型：像个“小学生”



仅能理解有限上下文

如同刚开始学语言的小学生，只能依赖非常局部的信息来预测下一个词，缺乏全局视野。

难以把握长程依赖关系

无法理解长句子中词与词之间的深层逻辑关联，对语言的整体语义把控能力较弱。

复杂任务表现笨拙

在处理需要深层理解的复杂语言任务时，预测结果往往不够准确，显得力不从心。

1.4 发展历史—预训练语言模型

预训练语言模型：AI的“语言预科班”



核心思想：预科班

预训练就像AI的“语言预科班”，为后续专业学习打下坚实基础。

学习海量通用知识

阅读海量文本数据，掌握语法规则、世界常识等通用语言能力。

任务迁移更高效

具备基础后，学习翻译、写作等具体任务时，效率大幅提升。

1.4 发展历史—预训练语言模型

第一步：海量阅读——预训练阶段



预训练数据：模型的“课本”

包括互联网上的网页、书籍、文章等海量文本数据，是模型学习语言的基础素材。

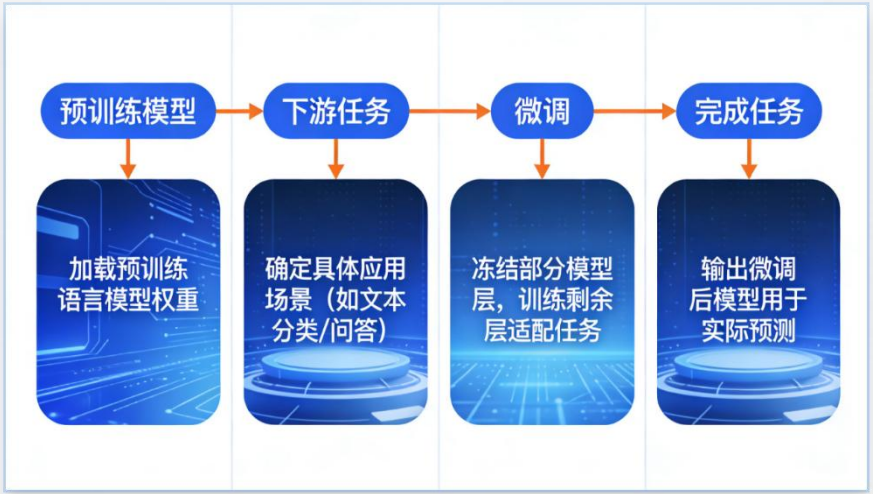


预训练任务：模型的“练习题”

如预测句子中缺失的词、判断句子是否通顺等。通过完成任务，模型掌握语言规律。

1.4 发展历史—预训练语言模型

第二步：知识迁移——微调阶段



预训练模型

已具备通用语言知识的基础模型，是微调的起点。

下游任务

具体的应用场景，如文本分类、机器翻译等实际需求。

微调 (Fine-tuning)

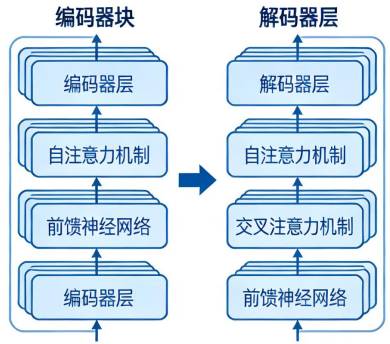
利用少量标注数据，调整模型参数以适配特定任务。

完成任务

模型快速掌握新技能，高效解决实际业务问题。

1.4 发展历史—预训练语言模型

Transformer架构：预训练模型的“大脑”



核心组成：编码器 + 解码器

编码器负责“理解”输入文本，解码器负责“生成”输出文本，两者协同工作构成模型的核心处理单元。

核心优势：高效并行处理

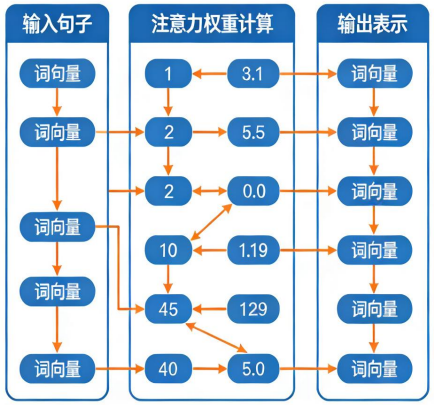
摒弃传统循环依赖，能够同时处理整个序列数据，大幅提升了模型的训练速度和计算效率。

能力突破：捕捉长距离依赖

通过自注意力机制（Self-Attention），模型能够直接关联文本中任意位置的词，更好地理解上下文语义。

1.4 发展历史—预训练语言模型

自注意力机制：模型的“专注力”



自注意力机制权重计算流程示意

核心组件：赋予模型“专注力”

自注意力机制是Transformer架构的核心，它让模型在处理序列数据时，能够聚焦于关键信息。

关联理解：建立词与词的联系

处理某个词时，模型会自动计算并关注句子中其他词与该词的关系权重，从而更深刻地理解上下文语义。

指代消解：精准识别代词指向

例如在“他把书放在桌子上，然后打开了它”中，模型能准确判断“它”指代的是“书”而非“桌子”。

1.4 发展历史—预训练语言模型

应用场景：智能问答



应用场景

广泛应用于智能问答系统，例如专业知识问答、企业智能客服、教育辅导问答等领域。

工作原理

模型基于预训练语言能力理解用户问题意图，从海量知识库中快速检索关联信息，生成精准、自然的答案。

核心优势

具备高响应速度与高准确率，能够7x24小时不间断提供专业知识服务，显著降低人力服务成本。

1.4 发展历史—预训练语言模型

应用场景：内容创作

AI 创作助手 - 输出预览

Prompt: 请创作一首关于科技与未来的现代诗

--- Generated Content ---

“指尖划过量子的尘埃，
代码编织着银河的纽带。
预训练的模型在数据中醒来，
为未来写下第一行告白。”

工作原理

用户输入提示词（Prompt），预训练语言模型基于海量数据学习到的模式，生成符合语境和要求的原创文本内容。

核心优势

能够瞬间生成高质量初稿，极大缩短创作周期；支持文章、诗歌、代码等多种体裁，有效激发创作者的灵感。

适用范围：智能写作辅助、代码自动补全、营销文案生成、创意故事续写等。

1.4 发展历史—预训练语言模型

总结：预训练语言模型的核心价值

高效学习

通过海量数据预训练，快速学习通用语言知识，奠定坚实基础。

强大迁移

能够将学到的通用知识灵活迁移到各种下游任务，适应能力极强。

广泛应用

在自然语言处理的各个领域都有重要应用，推动了AI技术的爆发式发展。



图示：预训练语言模型核心价值体系

1.4 发展历史—预训练语言模型

展望：未来的发展方向

多模态融合

结合图像、语音等多种信息，打破单一模态限制，实现更全面的理解和自然交互。

更高效的模型

通过模型压缩与架构优化，实现更小体积、更快推理、更强性能，显著降低计算成本。

更安全、更可控

着力解决偏见、幻觉与隐私问题，建立可信AI体系，确保技术发展的安全性与可靠性。



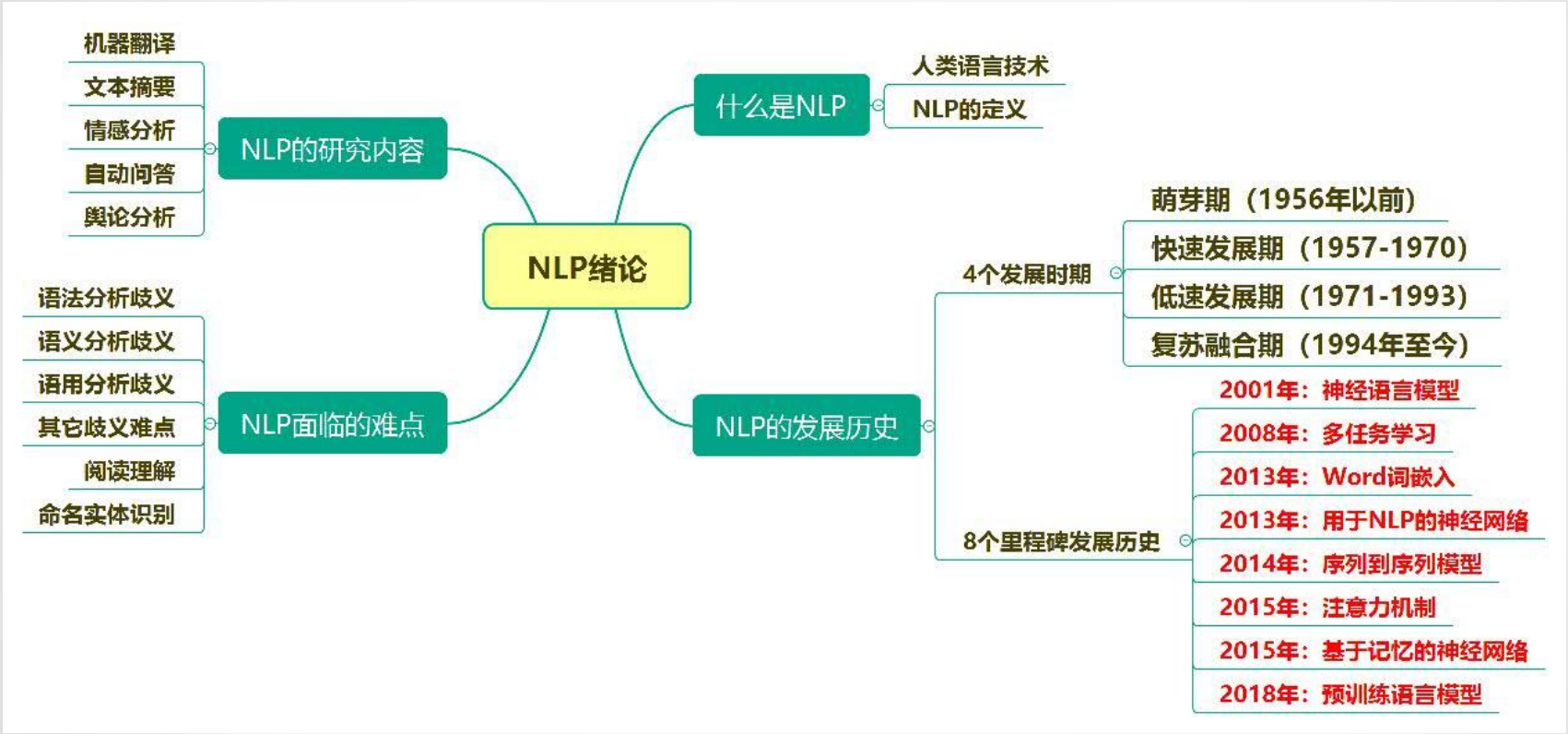
AI技术演进路线与愿景

本章教学内容

本章内容一览表

每小节概要	每小节知识点	重难点
第1小节　什 么 是 NLP	自然语言处理基本概念	是
第2小节　NLP的研究内容	机器翻译、文本摘要、情感分析、自动问答	否
第3小节　NLP面临的难点	歧义问题、标注问题、语料库缺乏问题	否
第4小节　NLP的发展历史	4个发展时期、8个里程碑发展历史	是
第5小节　教 学内 容 小结	总结本次课的重点难点知识	是
第6小节　课堂作业和讨论	课堂讨论	否

教学内容小结


NLP绪论
什么是NLP
人类语言技术
NLP的定义
NLP的研究内容
机器翻译
文本摘要
情感分析
自动问答
舆论分析
NLP面临的难点
语法分析歧义
语义分析歧义
语用分析歧义
其它歧义难点
阅读理解
命名实体识别
NLP的发展历史
4个发展时期
萌芽期（1956年以前）
快速发展期（1957-1970）
低速发展期（1971-1993）
复苏融合期（1994年至今）
8个里程碑发展历史
2001年：神经语言模型
2008年：多任务学习
2013年：Word词嵌入
2013年：用于NLP的神经网络
2014年：序列到序列模型
2015年：注意力机制
2015年：基于记忆的神经网络
2018年：预训练语言模型

本章教学内容

本章内容一览表

每小节概要	每小节知识点	重难点
第1小节　什 么 是 NLP	自然语言处理基本概念	是
第2小节　NLP的研究内容	机器翻译、文本摘要、情感分析、自动问答	否
第3小节　NLP面临的难点	歧义问题、标注问题、语料库缺乏问题	否
第4小节　NLP的发展历史	4个发展时期、8个里程碑发展历史	是
第5小节　教 学内 容 小结	总结本次课的重点难点知识	是
第6小节　课堂作业和讨论	课堂讨论	否

谢谢聆听！